

DATA VISUALIZATION

FINDING THE FINGERPRINT OF DATA

Data visualization is the one of the modern work horses of data science. Why is this so important? The simple reason is that, if the dimensionality of the data becomes too large, then the data might be difficult to visualize in a traditional way. Well, what is meant by this?

Suppose that you are working at a telecom operator and you are tasked with the responsibility of finding out how the telecom service can be more profitable. Recently, a new competitor entered into the market. However, the telecom operator still wants to be competitive and wants to find out how they can keep clients.

To make this task easier the telecom operator provides information about every client they recently served or are serving. The information the operator keeps from each customer are features. These can be the number of call failures, subscription length, seconds of use, frequency of use, frequency of SMS, distinct called numbers, if it is a pay-to-go or contract, amount of data usage, age of the customer and ultimately if they recently changed provider or are still active.

This complete collection of client-information is a dataset. One instance/datapoint of this dataset is hence a **client** and every client **has some features**. The amount of features is called the **'dimension' of the data**.

A **datapoint or instance** P corresponding to a certain observed instance with **measured features** $p_1, \cdots, p_n$ can be presented in the following way:

$$P = [p_1 \ p_2 \ p_3 \ p_4 \cdots \ p_{n-2} \ p_{n-1} \ p_n]$$

Note that this datapoint has dimension n. This is just one datapoint, so this means that the **complete dataset is a matrix**. For N clients the complete data-matrix looks as follows:

$$\boldsymbol{P} = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \\ \vdots \\ P_N \end{bmatrix} = \begin{bmatrix} p_{1,1} & p_{1,2} & p_{1,3} & p_{1,4} \cdots p_{1,n} \\ p_{2,1} & p_{2,2} & p_{2,3} & p_{2,4} \cdots p_{2,n} \\ p_{3,1} & p_{3,2} & p_{3,3} & p_{3,4} \cdots p_{3,n} \\ & & \vdots & \\ p_{N,1} & p_{N,2} & p_{N,3} & p_{N,4} \cdots p_{N,n} \end{bmatrix}$$

To get back to the client example, one will try to see if there can be **predicted** based on the other features that a client will possibly leave the deal or will stay. Hence, one needs to **classify** a customer in 'leaving' or 'staying'.

This is a task that can be done by a support vector machine or neural network. A full explanation of these deserves an article on its own. One must remember that a support vector machine (SVM) is a very reliable tool for binary classification, where one typically classifies in class 0 or class 1, or in this case in class 'client left the deal' or 'client stayed in the deal'. A neural network can theoretically do the same thing as a SVM. Both have their pros and cons.

For now, the goal is to **distinguish leaving clients from present clients**. In this way, there can be discovered which things in the service need to be improved to remain competitive.

First, we need to discover if there are serious differences between leaving clients and present clients before looking into improving services. Enter visualisation.

HIGH DIMENSIONALITY

Humans live in a three-dimensional world. This means that we can perfectly visualize lines, surfaces and volumes. However, **anything with higher dimension we cannot really imagine**. How does something look like in 4D? This seems more difficult.

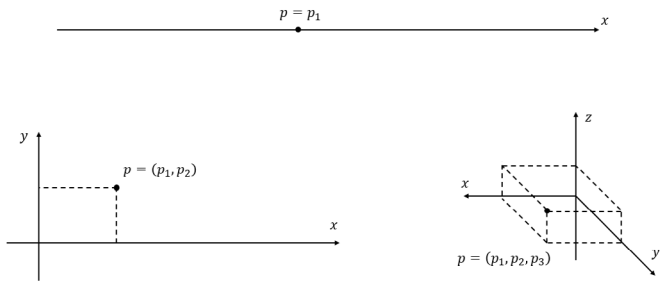


Figure 1: A single number can be represented on a line, two numbers on a surface and three numbers in a volume, but what with four numbers or more?

Hence, if we have a collection of datapoints with a higher dimension than three, **we cannot visualize it in a traditional way.** If the data cannot be visualized in the traditional technique due to the fact there are more than three features, then we have difficulties. It will be much harder to see if there is a pattern that can be exploited or two (or more) classes that can be easily separated.

FINDING A SOLUTION

A first proposal is **projecting the high-dimensional data on a line**. The problem with this method however is that datapoints which are far away from each other can be projected close to each other in the lower-dimension representation. This is not something which is desired because it does not give a reliable interpretation of the data.

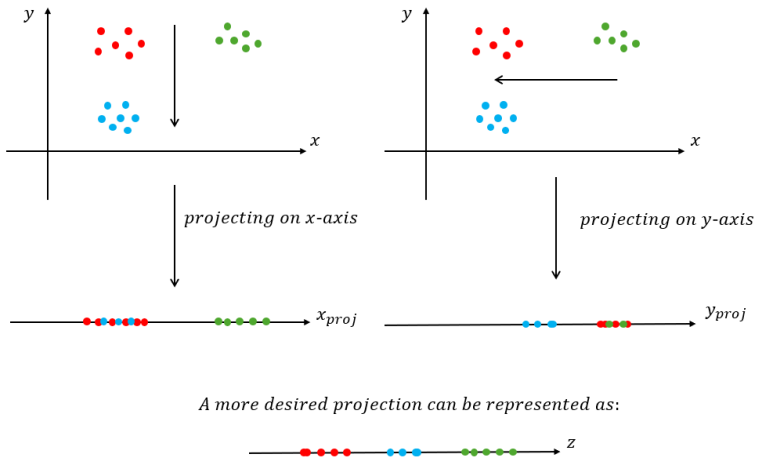


Figure 2: When projecting on the x-axis and the y-axis, it is clear that lower-dimensional representation is retrieved, which does not give a reliable representation of the data. Something more desired is shown below x_{proj} and y_{proj}.

t-SNE (t-distributed stochastic neighbour embedding)

Enter t-SNE**, a technique used in general to visualize high-dimensional datasets**. Let's start with a similar dataset as shown in figure 2. Then after running t-SNE in MATLAB the following is achieved:

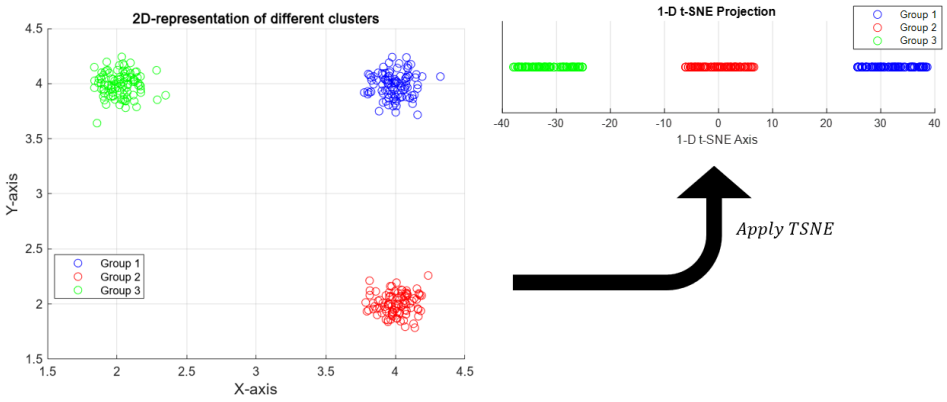


Figure 3: original dataset (left), 1D-dataset after running t-SNE (right).

How is the 1D-representation of the 2D-data achieved? First, the different datapoints are drawn on a 1D-line in random order. Note that since this is determined randomly. Hence, t-SNE will give a different representation for each time one performs it. Then the position of each datapoint is updated so that it is attracted to points close to it in the original dataset and it is repelled from datapoints which are far from the point in the original dataset.

First, there needs to be found out how much each point is similar to another point. This is done using a function g which used the distance between two points p_i and p_j in the given space:

$$s_{ij} = g(\|p_i - p_j\|)$$

What we want here is a value s_{ij} that is between 0 and 1 and represents **how similar points are**. Here 0 means that i and j are unsimilar and 1 means that they are the same point. Hence, something presented below is desirable:

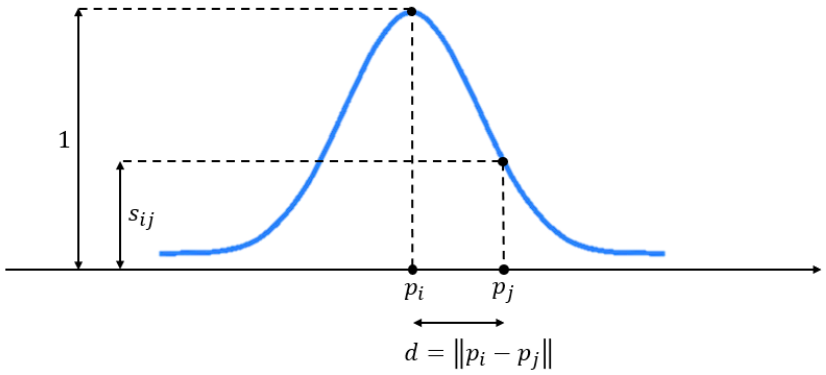


Figure 4: What is desired is that points close to p_i express high similarity. Points further from p_i express smaller similarity.

It is important to note that there are still some things to consider. For example, what if there are clusters which are **less dense**? Then it might be that these are underrepresented in the final representation.

In addition, if this rule is applied then there should hold that $s_{ij} = s_{ji}$. It should make sense that the similarity of point i to j is the same as the similarity of the point j to i.

To counteract the effect of less dense clusters one first calculates the following:

$$s_{ij} = \frac{g(\|p_i - p_j\|)}{\sum_{i \neq k} g(\|p_i - p_k\|)}$$

In this way, clusters with different densities will be taken into account. In this way there holds $\sum_{j=1, i \neq j}^{N} s_{ij} = 1$.

BEHIND THE CURTAINS

When one then looks at the paper of t-SNE, it turns out that the similarity is calculated as:

$$s_{ij} = \frac{\exp\left(-\|p_i - p_j\|^2 / 2\sigma_i^2\right)}{\sum_{i \neq k} \exp(-\|p_i - p_k\|^2 / 2\sigma_k^2)}$$

The function g we use here is a Gaussian. Why do we use this function? Well, it is a **good measure to see if two points are similar,** and it completely satisfies the desired shape presented in figure 4, where it is represented in 1D. However, it can be applied for points in any dimension, and that makes this function so powerful.

Using this, it turns out that the demanded symmetry is not present anymore. To counteract this, one then performs:

$$S_{ij} = \frac{s_{ij} + s_{ji}}{2n}$$

Here n is the amount of datapoints present in the dataset.

After calculating all the similarities between the points one can obtain the following matrix $\underline{S}$.

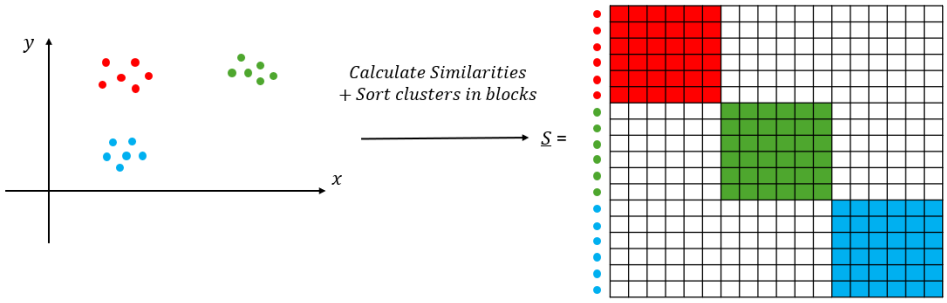


Figure 5: Clusters can be found when ordering the similarities in blocks.

Since the similarities are large between points in the same clusters, they can be ordered in these blocks. Thus, each block represents a cluster. The white values represent values which are smaller than the coloured values. This means that the similarity between different clusters is much smaller than the similarity between points in the same cluster.

Getting the lower-dimensional representation

Now that this step is done. It is time to **map from a larger dimension to a smaller dimension**. As mentioned before, all points are randomly initialized in the lower dimension.

We will randomly distribute the datapoints on a line as in the case for figure 4 (or surface/volume if desired) and shift the points along the space until a desired outcome in the lower-dimensional representation is achieved.

First, we start measuring the distance in the lower dimension to construct a similarity measure c_{ij} in the lower dimension between de points q_i and q_j using the similarity function h.

$$c_{ij} = \frac{h(\|q_i - q_j\|)}{\sum_{i \neq k} h(\|q_i - q_k\|)}$$

Since the points are all randomly initialized, the similarity matrix C won't look anything like the nice matrix, so we need to shift each point at the time so that it looks more like the original matrix $\underline{S}$. This is similar like a **puzzle that needs to be solved**. We need to shift each point at the time so that in the end we end up with the same distribution as in the original dimension.

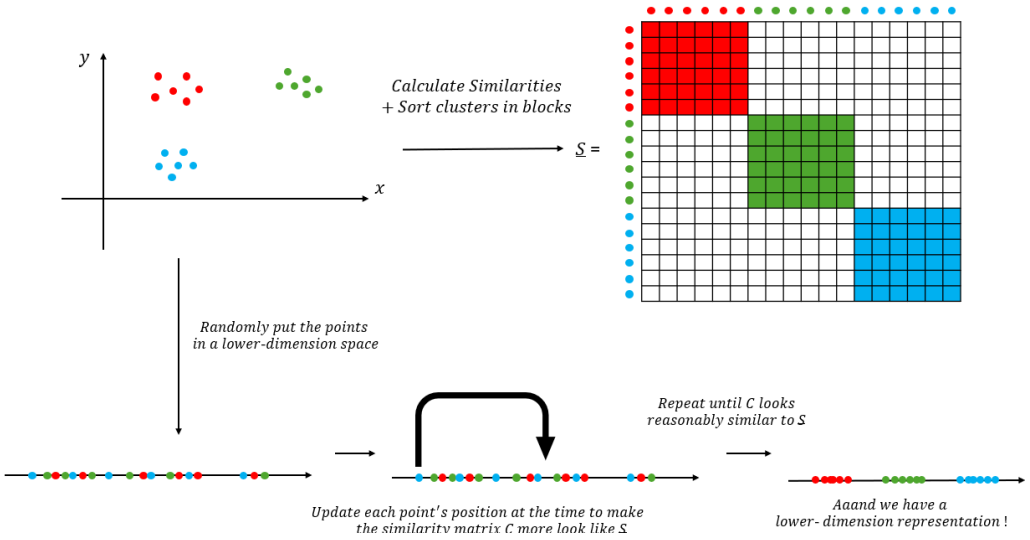


Figure 6: complete overview of the process.

There is one sidenote here. Note that it was left unclear how h could be chosen. One could argue that again the Gaussian function could be chosen. This would give the SNE-algorithm. However, **there is a good reason why we would use a different kind of similarity measure**.

If we would use a Gaussian function, then the low-dimensional representation might be too crowded. The different clusters would all be clumped together, and the visualization would not be desirable.

Why this is the case can be explained through an analogy. Imagine you have a huge room, like a ballroom, where all your friends are hanging out. In this big room, there's plenty of space, so your friends can spread out. Some are chatting in small groups, some are standing alone, and some are far away from each other.

Now, imagine you want to move all your friends into a much smaller room. In this smaller room, you still want to show who was close to who in the big room. But because the room is much smaller, everyone has to stand closer together.

The problem is, when you squish everyone into the small room, **you can't keep the same amount of space between them as they had in the big room**. Even though two friends were far apart in the ballroom, they might end up standing next to each other in the small room because there's not enough space. This makes it hard to show the same arrangement of friends as in the big room, and that's the "**crowding problem**". In a lower-dimensional space there is simply less space.

Hence, we need to make sure that things spread out and to do this we use a distribution with heavier tails namely the student-t distribution.

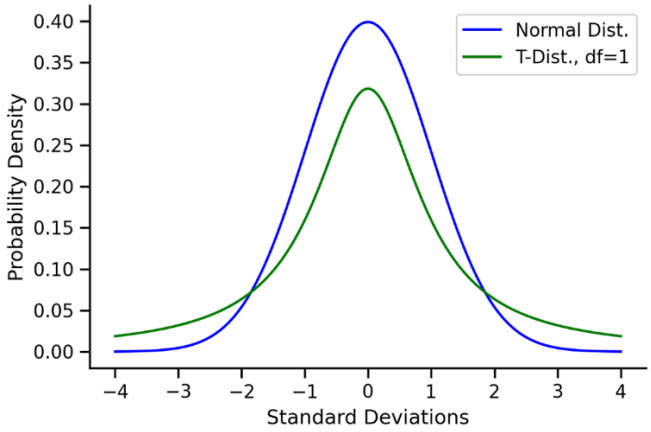


Figure 7: The tails of the student t-distribution are larger than the tails of a Gaussian distribution.

Imagine you have a special rule in the small room: friends who were far apart in the big room still need a bit more space around them, even if everyone has to get closer together. The heavy tails of the Student-t distribution act like this special rule, **making sure that distant friends don't get too crowded together in the small room**. This helps preventing the crowding problem, where everyone would otherwise end up too close to each other, making it hard to see the original relationships.

Hence, in the end the following is done. Note that this similarity function h is similar to the desired shape of a similarity function presented in figure 5.

$$c_{ij} = \frac{\left(1 + \|q_i - q_j\|^2\right)^{-1}}{\sum_{i \neq k}(1 + \|q_i - q_k\|^2)^{-1}}$$

RESULTS OF t-SNE

Now that we went through all the dominant aspects of t-SNE, it is important to see how it does for some cases. Suppose the following dataset:

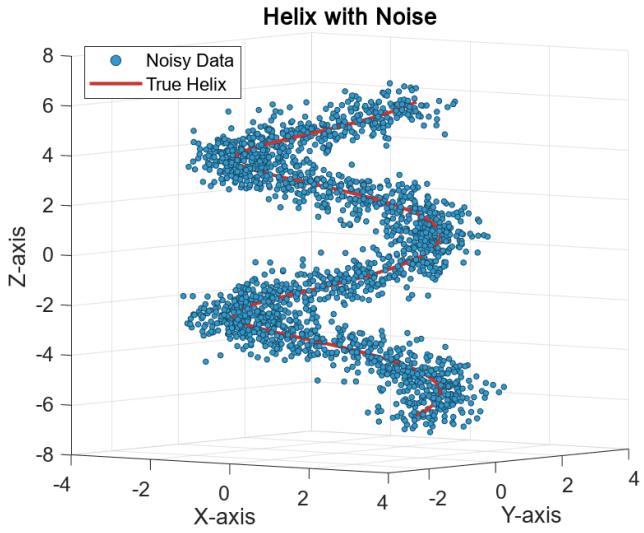


Figure 8: The datapoints of a dataset make up some helix-shape.

Now suppose that we have an extra feature for this dataset that is just a random variable. Think of this as a feature that has no further correlation with the other features.

However, this extra dimension will only contain noise and hence, the general structure of the dataset is preserved. But it might make it harder for t-SNE to make sense of the data. Note that it must be important in our case that t-SNE operates well under these circumstances since **there might be features in our client database that do not indicate if a client will leave**. Do you have an idea about how the 3D-representation of the changed dataset will be?

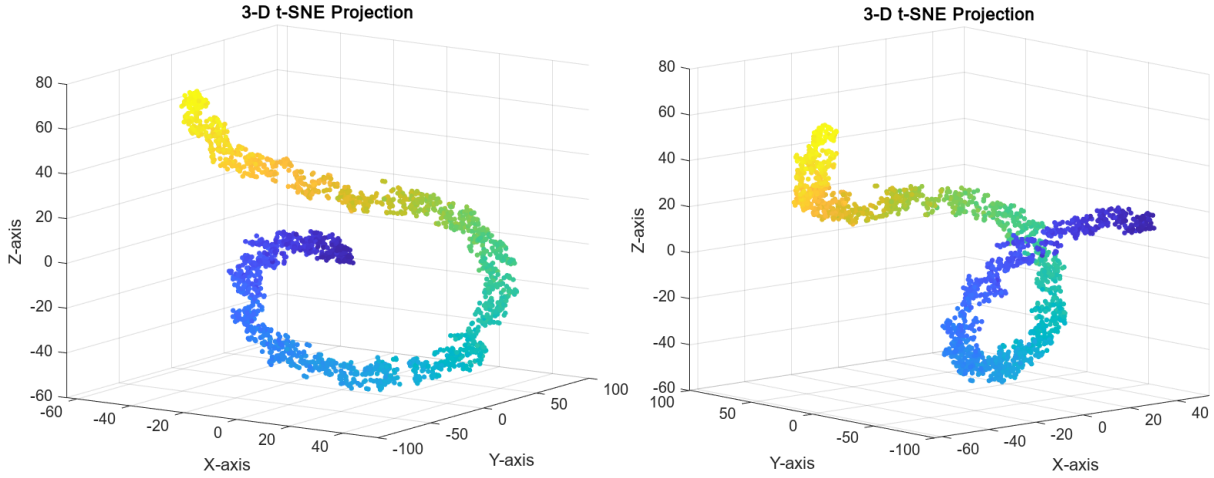


Figure 9: the 3D-result after applying t-SNE.

Note that the colour represents the original position of the datapoint in the curve. Hence, it is clear that t-SNE does a good job at recognizing that there is a line of datapoints for the helix.

Note that without background knowledge about the dataset, we have no intuition about the result we will get. We can only form conclusions based on the result t-SNE gives us.

Now suppose t-SNE gives the following 2D-result for our client dataset.

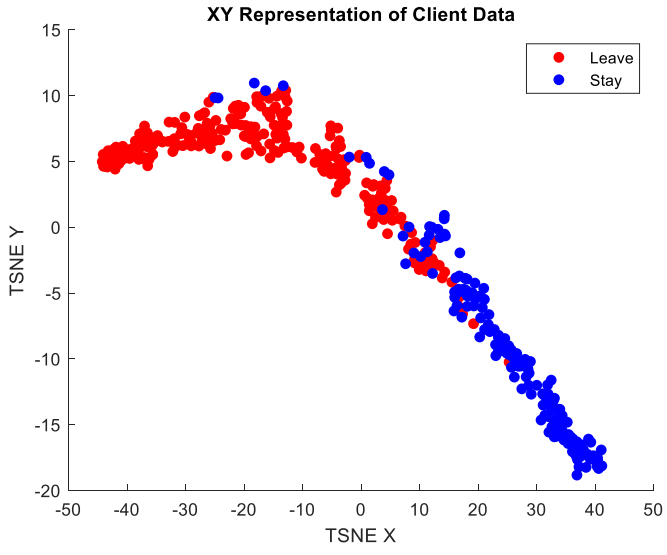


Figure 10: t-SNE output based on client dataset.

Based on this representation, it seems that a neural network or SVM might have an easy time in distinguishing leaving clients from staying clients. Hence, it might be easy to find how we can make clients stay. This is nice because now our boss can be assured that the problem can be solved given the client information present in the database.

Now assume that after applying t-SNE on the client dataset, the result would be the following:

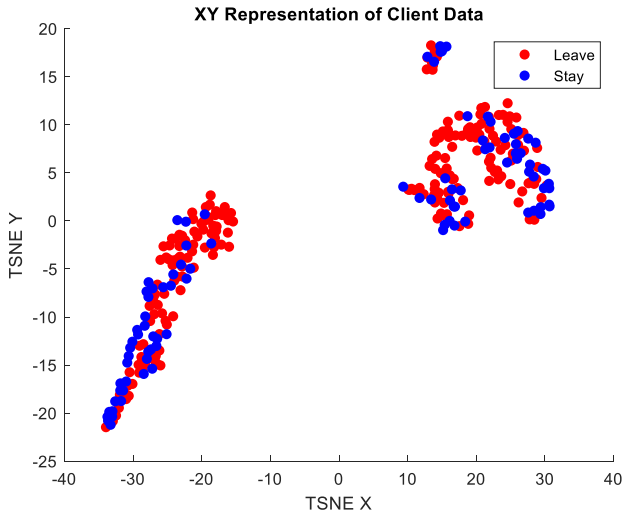


Figure 11: t-SNE output based on a different client dataset.

Since there is no great distinction between leaving or staying clients, it might be more difficult to find out how service can be improved. Hence, this might be an indication that the present client information is not sufficient to reliably estimate whether clients will leave or stay.

CONCLUDING t-SNE

In this article, it was observed that for **high-dimensional data it is difficult to get a feel for how it actually behaves**. This might make any objective in data science hard. A possible option is using algorithms that transform higher dimension data to a lower dimension. **The t-SNE algorithm is a good option for this, capturing the local and some global information by matching a lower-dimension distribution to a higher one**.

There are still some things that were left out. For example, how t-SNE exactly updates the position of the points in the lower-dimensional representation. Under the hood t-SNE will try to make sure that the distribution between points in the higher-dimension will align with the distribution of points in the lower-level embedding.

An alternative to t-SNE is UMAP. UMAP can be generally applied and deserves an article on its own. UMAP can be faster than t-SNE and often captures more of the global relationships, which can help in understanding the broader patterns in a complex dataset.

Sources:
https://www.youtube.com/watch?v=NEaUSP4YerM&ab_channel=StatQuestwithJoshStarmer
https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf
https://medium.com/@aeonaten/understanding-umap-uniform-manifold-approximation-and-projection-cede51c477d9
https://towardsdatascience.com/t-sne-clearly-explained-d84c537f53a
https://distill.pub/2016/misread-tsne/

*Dit is een gastbijdrage van **Jasper Visterin** voor SAI Update. Dit artikel schreef hij in het kader van zijn master-opleiding als data-ingenieur, en meer bepaald burgerlijk ingenieur wiskundige ingenieurstechnieken aan de KU Leuven.*